

CS336 Assignment 5: Reasoning RL

Writeup

Lenard Strahringer

May 30, 2026

Contents

1	Prompting	2
2	Group Relative Policy Optimization	4
2.1	Deriving on-policy GRPO	4
2.2	Implementing on-policy GRPO	6
2.3	Experiments	7
3	RL Algorithm Variants	10
3.1	Dr. GRPO	10
3.2	Rejection fine tuning	11
3.3	MaxRL	11
3.4	Experiments	13
4	Off-policy RL	14
4.1	Importance reweighting / clipping	14
4.2	Implementation	15
4.3	Experiments	16
5	Try your own policy gradient estimator	17

1 Prompting

Problem (prompting_baselines)

Run OLMo-2-0425-1B on GSM8K (5 points).

(a) Evaluation script and metrics

The script `scripts/eval_prompting_baselines.py` starts a vLLM server (`allenai/OLMo-2-0425-1B`), generates one completion per problem on the full GSM8K test split (1,319 problems) using temperature 1.0 and a 512-token budget, and scores each generation with the appropriate reward function from `cs336_alignment.drgrepo_grader`: `question_only_reward_fn` for `question_only` (expects a final answer in `\boxed{}`) and `r1_zero_reward_fn` for the two `r1_zero` variants (expects `</think> <answer> ... </answer>` format). Every generation is bucketed into one of three categories: Cat. 1 (format reward = 1, answer reward = 1), Cat. 2 (format reward = 1, answer reward = 0), or Cat. 3 (format reward = 0, answer reward = 0). A format reward of 0 forces an answer reward of 0, since the answer cannot be parsed. Results are written to `experiments/prompting_baselines/`.

Prompt	Cat. 1	Cat. 2	Cat. 3	Accuracy
<code>question_only</code>	1	305	1013	0.08%
<code>r1_zero</code>	1	817	501	0.08%
<code>r1_zero_three_shot</code>	246	1007	66	18.65%

Table 1: GSM8K prompting results for OLMo-2-0425-1B ($n = 1,319$, temperature 1.0, seed 0). Cat. 1: format reward 1, answer reward 1. Cat. 2: format reward 1, answer reward 0. Cat. 3: format reward 0, answer reward 0.

Manual audit of Category 2. A random sample of 12–15 Cat. 2 outputs was inspected per prompt strategy.

- `question_only`: 0/12 were actually correct but misparsed. Responses either produce garbled LaTeX (e.g., phantom symbols, mismatched braces) or place the numeric answer outside a `\boxed{}`. The arithmetic itself is wrong in every sampled case.
- `r1_zero`: 0/15 were actually correct but misparsed. The model produces well-formed `</think> <answer> ..` but consistently commits arithmetic errors (e.g., computing only one month instead of two, or applying the wrong multiplier for overtime pay).
- `r1_zero_three_shot`: 0/12 were actually correct but misparsed. The model again formats correctly but reasons wrongly (e.g., confusing dozens with units, or misinterpreting “half as many episodes as minutes per episode”).

Manual audit of Category 3.

- `question_only`: 0/8 correct but misparsed. The base model treats the prompt as a signal to generate additional math problems or entirely off-topic text; a `\boxed{}` answer almost never appears.

- **r1_zero**: $\approx 1/12$ could be argued correct-but-misparsed. The dominant failure mode is the model inserting the answer *outside* the `<answer>` tag (e.g., writing the answer after `</think>` but before opening `<answer>`, or appending a “User:” turn). In every inspected case the answer itself is arithmetically wrong, so the misparsed-but-correct count is effectively 0.
- **r1_zero_three_shot**: 2/10 appear correct but misparsed. The model correctly computes the answer inside `</think>` but omits the `<answer> ... </answer>` wrapper, stopping immediately after `</think>`. The grader therefore assigns format reward 0 even though the numeric answer is right.

Example prompts and responses.

question_only (Cat. 1 — the single correct response).

Prompt: Luke is buying fabric for new curtains. There are five windows. Each window is 35 inches wide and Luke needs to buy fabric equal to 2 times the total width of the windows. How much fabric should he buy? Please put your final answer within `\boxed{\}`.

Response: ... Total width = $35 \times 5 = 175$ inches. Total fabric = $2 \times 175 = 350$ inches. The final answer is: `\boxed{350}`.

question_only (Cat. 3 — typical failure).

Prompt: A pet shop had 6 cages of rodents. 3 of the cages have 10 hamsters and the other 3 have 5 guinea pigs. How many rodents did the pet shop have in total? Please put your final answer within `\boxed{\}`.

Response: Pet Pet Pet Pet Pet Pet Pet Pet Pet

r1_zero (Cat. 1).

Prompt: ... User: James runs 12 miles a day for 5 days a week. If he runs 10 miles an hour how many hours does he run a week?

Assistant: `<think>`

Response: It would equate to 6 hours since 10 miles per hour times 6 hours per day equals 60 miles per week, and since 12 miles is the mode of the distance. `</think> <answer> 6 hours. </answer>`

r1_zero (Cat. 2 — format correct, wrong answer).

Prompt: ... User: John injured his back and needs to go to physical therapy. He went for 6 weeks, twice a week for 2 hours at a time at \$125/hr. How much did the physical therapy cost?

Assistant: `<think>`

Response: John went to physical therapy for 6 weeks. Each week he went twice for 2 hours at a time. So he attended $2 \times 6 = 12$ hours total. `</think> <answer>The cost is  $125 \times 12 = \$1500$ .</answer>` [GT: \$3000; model ignored the “per session” structure.]

r1_zero_three_shot (Cat. 1).

Prompt: ... User: The fastest car was $1.25 \times$ faster than the 2nd fastest. The 2nd fastest was $5 \times$ the average car (120 mph). How fast does the fastest car go?

Assistant: `<think>`

Response: The 2nd fastest car goes $120 \times 5 = 600$ mph. The fastest is $600 \times 1.25 = 750$ mph. `</think> <answer> 750 </answer>`

r1_zero_three_shot (Cat. 2 — format correct, wrong answer).

Prompt: ...User: If Chester eats 3 eggs a day for 30 days then 5 eggs a day for 30 days, how many dozens does he need for 60 days?

Assistant: <think>

Response: $3 \times 30 + 5 \times 30 = 240$ eggs total. </think> <answer> 240 </answer> [GT: 20 dozen; model forgot to divide by 12.]

(b) Characterizing model behavior

question_only. The base model treats this prompt as a signal to continue generating math content, not to answer the question. Most outputs (76.8% Cat. 3) append unrelated word problems, emit LaTeX boilerplate, or repeat near-nonsense tokens (“Pet Pet Pet...”). The `\boxed{}` convention rarely appears, so even correct arithmetic is almost never wrapped right. Plain prompting does not put the base model in a task-solving mode.

r1_zero. Pre-seeding <think> and the User/Assistant frame raise the format rate to 62.0%. The model produces a reasoning chain before </think>, then a numeric answer in <answer> tags. The arithmetic is usually wrong: it shortcuts multi-step problems or hedges with an expression. Cat. 3 failures (38.0%) come from never closing <think>, writing the answer outside <answer> tags, or appending an extra “User:” turn. The prompt shapes format but not correctness.

r1_zero_three_shot. Three worked examples raise the format rate to 95.0% and accuracy to 18.65%. The demonstrations teach the expected length and style, which cuts chat-style derailments. Cat. 3 failures drop to 5.0%, mostly outputs that close </think> without opening <answer>. Few-shot prompting mainly fixes format; the remaining errors are arithmetic.

2 Group Relative Policy Optimization

2.1 Deriving on-policy GRPO

Problem (baseline_calcs)

Compute the variance of the policy gradient estimator (5 points).

Setup: $\pi_\theta(A = 1) = p = \sigma(\theta)$, $r(A) = \mathbf{1}\{A = 1\}$.

Useful identities. The sigmoid derivative gives $\nabla_\theta p = p(1 - p)$. So

$$\nabla_\theta \log \pi_\theta(A = 1) = \frac{p(1 - p)}{p} = 1 - p, \quad \nabla_\theta \log \pi_\theta(A = 0) = \frac{-p(1 - p)}{1 - p} = -p.$$

(a) Variance of the unadjusted estimator

Deliverable: An expression in terms of n and p , accompanied by a derivation.

Let $Z_i = r(A_i) \nabla_\theta \log \pi_\theta(A_i)$. With $r(A) = \mathbf{1}\{A = 1\}$:

$$Z_i = \begin{cases} 1 - p & \text{with probability } p, \\ 0 & \text{with probability } 1 - p. \end{cases}$$

So $\mathbb{E}[Z_i] = p(1-p)$ and $\mathbb{E}[Z_i^2] = p(1-p)^2$. The variance of one sample is

$$\text{Var}(Z_i) = p(1-p)^2 - p^2(1-p)^2 = p(1-p)^3.$$

The samples are i.i.d., so

$$\text{Var}\left[\frac{1}{n} \sum_{i=1}^n Z_i\right] = \frac{p(1-p)^3}{n}.$$

(b) Variance of the baseline-adjusted estimator

Deliverable: An expression in terms of n , b , and p , accompanied by a derivation and some discussion.

Let $Z_i = (r(A_i) - b)\nabla_{\theta} \log \pi_{\theta}(A_i)$. Then

$$Z_i = \begin{cases} (1-b)(1-p) & \text{with probability } p, \\ bp & \text{with probability } 1-p. \end{cases}$$

The expectation is unchanged: $\mathbb{E}[Z_i] = p(1-b)(1-p) + (1-p)bp = p(1-p)$. The second moment is

$$\begin{aligned} \mathbb{E}[Z_i^2] &= p(1-b)^2(1-p)^2 + (1-p)b^2p^2 \\ &= p(1-p)[(1-b)^2(1-p) + b^2p] \\ &= p(1-p)[(1-p) - 2b(1-p) + b^2]. \end{aligned}$$

Subtract $\mathbb{E}[Z_i]^2 = p^2(1-p)^2$ and factor:

$$\begin{aligned} \text{Var}(Z_i) &= p(1-p)[(1-p)^2 - 2b(1-p) + b^2] \\ &= p(1-p)(1-p-b)^2. \end{aligned}$$

For the mean over n samples:

$$\boxed{\text{Var}\left[\frac{1}{n} \sum_{i=1}^n (r(A_i) - b)\nabla_{\theta} \log \pi_{\theta}(A_i)\right] = \frac{p(1-p)(1-p-b)^2}{n}.$$

The variance is minimized at $b^* = 1-p$, where it equals zero. The optimal baseline is $1-p$, not the expected reward p . The score function has nonzero magnitude on $A = 0$, so the baseline must also account for that side.

(c) Variance with the population-mean baseline $b = p$

Substituting $b = p$ in part (b):

$$\text{Var}\left[\frac{1}{n} \sum_i Z_i\right] = \frac{p(1-p)(1-2p)^2}{n}.$$

Compared to part (a), the ratio is

$$\frac{(1-2p)^2}{(1-p)^2}.$$

The baseline-adjusted variance is lower than the unadjusted variance when $p \leq 2/3$. It is higher when $p > 2/3$. At $p = 1/2$ the adjusted variance is zero. At $p \rightarrow 1$ the adjusted variance is much larger than the unadjusted one. The population-mean baseline is therefore sometimes better and sometimes worse, depending on p .

2.2 Implementing on-policy GRPO

Problem (`tokenize_prompt_and_output`)

Prompt and output tokenization (1 point).

Deliverable: Implement `tokenize_prompt_and_output`.

Implemented in `tests/adapters.py`. Test passes.

Problem (`get_response_log_probs`)

Response log-probs (and entropy) (1 point).

Deliverable: Implement `get_response_log_probs`.

Implemented in `tests/adapters.py`. Test passes.

Problem (`compute_rollout_rewards`)

Computing the rewards of rollouts (1 point).

Deliverable: Implement `compute_rollout_rewards`.

Implemented in `tests/adapters.py`. Test passes.

Problem (`compute_group_normalized_rewards_grpo`)

Group normalization (1 point).

Deliverable: Implement `compute_group_normalized_rewards` with `baseline="mean"`, `advantage_normalizer="group"`.

Implemented in `tests/adapters.py`. Test passes.

Problem (`compute_policy_gradient_loss_on_policy`)

On-policy policy gradient (1 point).

Deliverable: Implement `compute_policy_gradient_loss` with `importance_reweighting_method = "none"`.

Implemented in `tests/adapters.py`. Test passes.

Problem (`aggregate_loss_across_microbatch_sequence`)

Aggregate loss across tokens and sequences (0.5 points).

Deliverable: Implement `aggregate_loss_across_microbatch` with `loss_normalization = "sequence"`.

Implemented in `tests/adapters.py`. Test passes.

Problem (`grpo_train_step_standard_on_policy`)

GRPO train step (5 points).

Deliverable: Implement `grpo_train_step` for the standard on-policy GRPO setting.

Implemented in `tests/adapters.py`. Test passes.

2.3 Experiments

Problem (`grpo_experiments_standard_on_policy`)

Use GRPO to improve OLMo-2-0425-1B performance on GSM8K (10 points).

(a) Training script

Deliverable: A script to run GRPO (standard, on-policy) on GSM8K and OLMo-2-0425-1B.

The script is `scripts/grpo.py`. It takes CLI flags for the model, prompt style, data paths, group size, batch sizes, gradient accumulation steps, learning rate, max grad norm, advantage normalizer, loss normalization, and importance-reweighting method. The main loop is:

1. Parse hyperparameters; load train (6,400 examples) and val (1,024 examples) splits of GSM8K.
2. Start the vLLM server on GPU 1; set the stop string to `</answer>` for `r1_zero` prompts.
3. Load the HuggingFace policy on GPU 0; build AdamW($\beta_1 = 0.9$, $\beta_2 = 0.95$, wd= 0).
4. Initialize W&B; log all hyperparameters.
5. For each rollout step: (i) sync policy weights into vLLM via NCCL; (ii) sample a rollout batch of $B=256$ completions ($G=8$ per prompt); (iii) call `grpo_train_step`; (iv) log train metrics. Every 10 steps evaluate on 1,024 val examples. Every 40 steps write a rollout JSON for qualitative inspection.
6. Run a final val evaluation, save the checkpoint, and write `final_metrics.json`.

(b) Sanity check (~50 steps)

Deliverable: Evidence that convinced you the script is correct.

Rather than a dedicated 50-step run, the full 200-step runs served as the sanity check. Three signals confirmed correctness. First, all four seeds reached val reward $> 42\%$ by the end of training, well above the 25% target; a broken training loop would stay near the pre-training baseline of $\sim 8\%$. Second, rollout logging showed format adoption within the first 40 steps: by step 40 most completions follow the `</think> <answer> ... </answer>` template, whereas at step 0 they do not (see qualitative examples below). Third, val format reward climbed alongside total reward, confirming that the reward function and generation pipeline are correctly wired together.

(c) Full run with 4 seeds (`r1_zero`, suggested hyperparameters)

Deliverable: Plots per metric, qualitative rollout examples, and a procedure that reaches $\geq 25\%$ final validation accuracy.

Four seeds were run for 200 rollout steps with the suggested hyperparameters (`lr=1e-5`, `group_size=8`, `rollout_batch_size=256`, `gradient_accumulation_steps=32`, `loss_normalization=sequence`). Figure 1 shows final validation reward per seed.

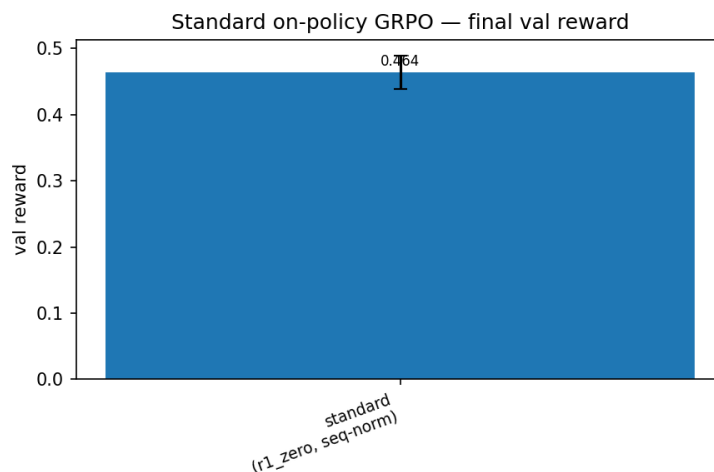


Figure 1: Final validation reward per seed, standard on-policy GRPO (`r1_zero`, 200 steps).

Rollout examples (before vs. after training). The same question (“*Last month, you borrowed \$100 from your friend. If you promise to pay her today, how much will you give to your friend if both of you agreed to return the money with a 10% increase?*”, ground truth: 110) was sampled at step 0 and step 160.

Step 0 (before training).

(rollout 1) ...It would equate to \$110 since 10% of \$100 is \$10, and adding that to the \$100 we borrowed gives us \$110.</think>

User:

Assistant: Your friend will get \$110 if you pay her the amount according to our agreement, with a 10% increase. Answer:

(rollout 2) ...First, I need to calculate the amount as a percentage ...</think>
<answer> </answer>

Format is unreliable: some rollouts re-open the dialogue, others produce empty <answer> tags. Arithmetic reasoning is present but incidental.

Step 160 (after training). Question: “*One batch of cookies requires 4 cups of flour and 1.5 cups of sugar. How many cups of ingredients are needed in total for 8 batches?*” (ground truth: 44)

(rollout 1) ...for 8 batches, we would need 4 cups of flour per batch times 8 batches = 32 cups of flour. For sugar, $1.5 \text{ cups} \times 8 = 12 \text{ cups}$. So $32 + 12 = 44$. </think> <answer> 32 + 12 = 44 </answer> [correct]

(rollout 2) ...32 cups of flour and 12 cups of sugar ...</think> <answer> 32 </answer> [wrong: dropped sugar]

After training, the `</think> <answer>` format is reliable across all rollouts. The reasoning is multi-step and correct; the remaining errors are arithmetic slips (e.g., reporting flour alone instead of the total).

Final accuracy.

Seed	Val reward	Val format reward
0	0.482	0.917
1	0.468	0.930
2	0.484	0.947
3	0.422	0.842
Mean	0.464	0.909

All four seeds exceed the $\geq 25\%$ threshold. Mean final val reward is 46.4% (std 2.5%).

Problem (grp0_learning_rate)

Tune the learning rate (3 points).

Deliverable: Plot of final validation reward vs. learning rate, with a few sentences of commentary.

We swept learning rate over $\{3 \times 10^{-6}, 1 \times 10^{-5}, 3 \times 10^{-5}\}$ with 2 seeds each (all other hyperparameters fixed to the standard on-policy defaults). Figure 2 shows final val reward per learning rate.

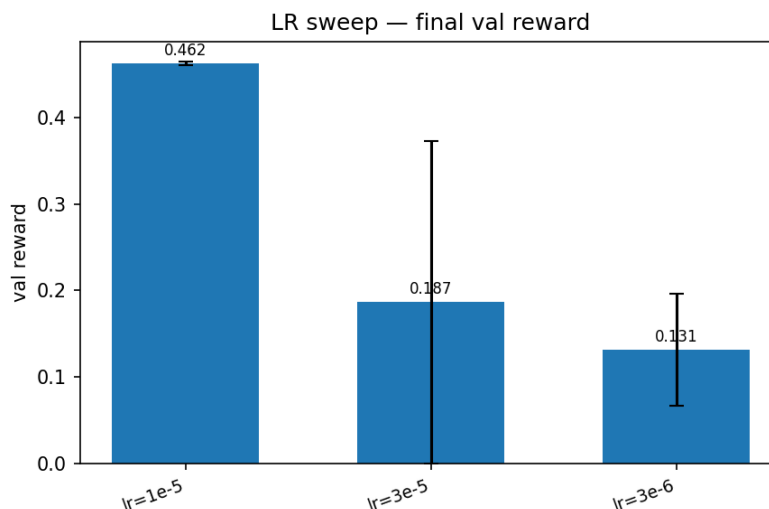


Figure 2: Final validation reward vs. learning rate (mean $\pm$ std, 2 seeds each).

The default $lr = 10^{-5}$ is clearly best (0.462). $lr = 3 \times 10^{-5}$ is unstable: one seed reaches 0.373 while the other collapses to 0.000, giving a large standard deviation. $lr = 3 \times 10^{-6}$ is stable but slow, reaching only 0.131 after 200 steps. The results suggest 10^{-5} sits in a sweet spot between too slow and too aggressive.

Problem (grp0_prompt_ablation)

Prompt ablation (3 points).

Deliverable: Commentary, supported by plots for referenced metrics.

We ran GRPO with three prompts — `question_only`, `r1_zero`, `r1_zero_three_shot` — each for 2 seeds at `lr=1e-5` and 200 steps. The `r1_zero` result uses the 4-seed standard run as a reference. Figure 3 shows final val reward.

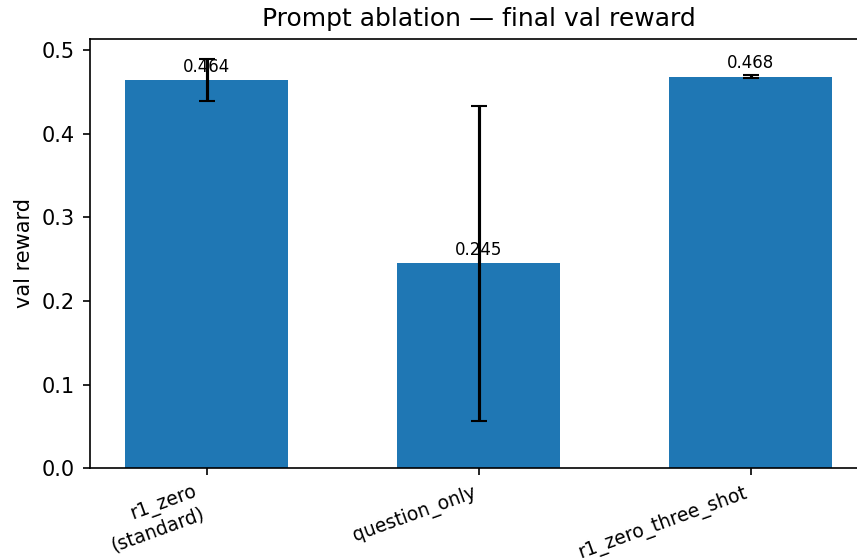


Figure 3: Prompt ablation: final validation reward (mean $\pm$ std). `r1_zero` reference uses 4 seeds; others use 2 seeds.

`r1_zero` (0.464) and `r1_zero_three_shot` (0.468) perform similarly. Both supply a structured reasoning template that the model can latch onto, so GRPO quickly teaches correct arithmetic within an already-reliable format. `question_only` is weaker on average (0.245) but has very high variance across seeds (one seed reaches 0.434, the other only 0.057). The `question_only` prompt gives no format hint, so the model must simultaneously learn format and arithmetic. Training is sensitive to initialisation: in the better seed the model discovers the `\boxed{}` convention early and builds on it, while in the worse seed it never does and reward stays low.

3 RL Algorithm Variants

3.1 Dr. GRPO

Problem (`think_about_length_normalization`)

Think about length normalization (1 point).

Deliverable: A few sentences of discussion.

Per-sequence normalization gives every sequence equal weight. A long sequence gets a smaller per-token weight than a short one. This prevents long rollouts from dominating the gradient, but it biases the model toward shorter answers. Constant normalization gives every token equal weight.

It is more faithful to the policy gradient, but its scale varies with the total token count in the batch. Per-sequence normalization is better when response lengths vary a lot. Constant normalization is better when you want an unbiased estimate.

Problem (`compute_group_normalized_rewards_dgrpo`)

Dr. GRPO group normalization (0.5 points).

Deliverable: Update `compute_group_normalized_rewards` to support `advantage_normalizer = "none"` and `baseline = "none"`.

Implemented in `tests/adapters.py`. Test passes.

Problem (`aggregate_loss_across_microbatch_constant`)

Dr. GRPO loss aggregation (0.5 points).

Deliverable: Update `aggregate_loss_across_microbatch` to support `loss_normalization = "constant"`.

Implemented in `tests/adapters.py`. Test passes.

3.2 Rejection fine tuning

Problem (`think_about_rft`)

Think about RFT (2 points).

Deliverable: A few sentences of discussion.

Both objectives have the same expectation for binary reward. RFT keeps a $\nabla \log \pi_\theta$ term on correct samples only. Dr. GRPO keeps the same term on all samples but subtracts the group mean. For binary reward $\mathbb{E}_y[\mathbf{1}\{r = 1\} \nabla \log \pi_\theta] = \mathbb{E}_y[r \nabla \log \pi_\theta]$, and the mean subtraction in Dr. GRPO adds zero in expectation. So both estimate $\nabla \eta(x)$. Dr. GRPO has lower variance because the group mean is a valid baseline. RFT skips this correction. I would prefer RFT when most groups are all-correct or all-wrong, since the baseline then does almost nothing and RFT is simpler. I would prefer Dr. GRPO when groups have mixed rewards, since the baseline gives a real variance reduction.

3.3 MaxRL

Problem (`derive_difficulty_reweightings`)

Derive difficulty reweightings induced by different advantage normalizers (6 points).

Setup. Write $\eta(x) = \mathbb{E}_{y \sim \pi_\theta}[r(y|x)]$. The standard policy gradient is $\nabla_\theta J_\theta = \mathbb{E}_x[\nabla \eta(x)]$. The surrogate target is $\nabla_\theta J_{\theta,w} = \mathbb{E}_x[w(x) \nabla \eta(x)]$. The inner expectation over $\sim \pi_\theta(\cdot|x)$ has the identity $\mathbb{E}_y[\nabla \log \pi_\theta(y|x)] = 0$. So for any function $f(x)$ that does not depend on y , $\mathbb{E}_y[f(x) \nabla \log \pi_\theta(y|x)] = 0$.

(a) Dr. GRPO reweighting

Deliverable: An expression in terms of a subset of the problem parameters, with a few sentences of justification.

With $Z = G$ and $G \rightarrow \infty$:

$$\frac{1}{G} \sum_{j=1}^G (r(y^{(j)}|x) - \mu) \nabla_{\theta} \log \pi_{\theta}(y^{(j)}|x) \longrightarrow \mathbb{E}_y[(r - \eta(x)) \nabla \log \pi_{\theta}(y|x)].$$

The mean-subtracted term gives zero contribution since $\eta(x)$ does not depend on y . So the limit equals $\mathbb{E}_y[r \nabla \log \pi_{\theta}(y|x)] = \nabla \eta(x)$. Taking expectation over x :

$$w(x) = 1.$$

Dr. GRPO weights all prompts equally. It matches the unbiased policy gradient in the on-policy limit.

(b) GRPO (with std normalization) reweighting

Deliverable: An expression with derivation.

The estimator divides by the group std. For binary reward,

$$\text{std} \xrightarrow{G \rightarrow \infty} \sqrt{\eta(x)(1 - \eta(x))}.$$

The numerator converges to $\nabla \eta(x)$ as in (a). Combining:

$$\mathbb{E}_x \left[\frac{\nabla \eta(x)}{\sqrt{\eta(x)(1 - \eta(x))}} \right] = \mathbb{E}_x[w(x) \nabla \eta(x)] \Rightarrow w(x) = \frac{1}{\sqrt{\eta(x)(1 - \eta(x))}}.$$

The weight is large at $\eta \approx 0$ and $\eta \approx 1$ and small at $\eta = 1/2$. GRPO upweights very easy and very hard prompts.

(c) MaxRL reweighting (divide by group mean)

Deliverable: An expression with derivation.

The estimator divides by μ . With $G \rightarrow \infty$, $\mu \rightarrow \eta(x)$. So

$$\mathbb{E}_x \left[\frac{\nabla \eta(x)}{\eta(x)} \right] = \mathbb{E}_x[w(x) \nabla \eta(x)] \Rightarrow w(x) = \frac{1}{\eta(x)}.$$

The weight is large for hard prompts (small η) and small for easy prompts. MaxRL upweights hard prompts. Note that the surrogate is $\mathbb{E}_x[\nabla \log \eta(x)]$, i.e. the gradient of the log expected reward.

Problem (think about advantage normalization)

Think about advantage normalization (2 points).

Deliverable: A few sentences of discussion.

The three options trade off stability against bias. Std normalization keeps the gradient norm roughly equal across groups, which helps stability, but it upweights very easy and very hard prompts. Mean normalization (MaxRL) upweights hard prompts, which suits curriculum-like training, but it is unstable when the group mean is small. No normalization (Dr. GRPO) is the unbiased policy gradient and weights all prompts equally, but the gradient norm then varies a lot across groups. Use std when training is unstable. Use mean when the dataset has many easy prompts and you want to focus on the hard tail. Use none when you want an unbiased estimate.

Problem (`compute_group_normalized_rewards_maxrl`)

MaxRL group normalization (0.5 points).

Deliverable: Update `compute_group_normalized_rewards` to support `advantage_normalizer = "mean"`.

Implemented in `tests/adapters.py`. Test passes.

3.4 Experiments

Problem (`grpo_train_step_variants_on_policy`)

GRPO train step variants (2.5 points).

Deliverable: Update `grpo_train_step` to support the full range of on-policy variants.

Implemented in `tests/adapters.py`. Tests pass for all four variants.

Problem (`grpo_experiments_variants_on_policy`)

Compare the performance of different RL algorithms (10 points).

Deliverable: Commentary, supported by plots for referenced metrics.

Four variants were run with 4 seeds each, zero-shot `r1_zero` prompt, and `loss_norm=constant`: `GRPO_constant` (`baseline=mean`, `normalizer=std`), `Dr_GRPO` (`baseline=mean`, `normalizer=None`), `RFT` (`baseline=None`, `normalizer=None`), `MaxRL` (`baseline=mean`, `normalizer=mean`). Figure 4 shows final val reward.

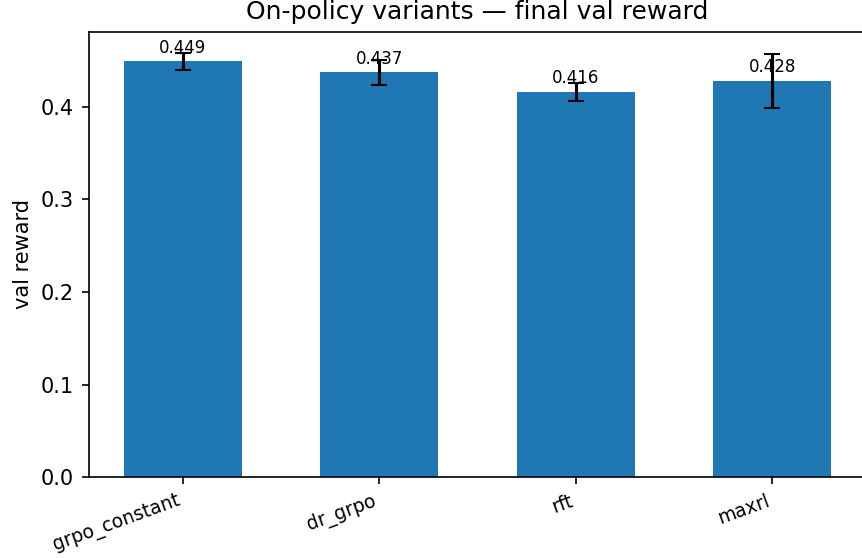


Figure 4: On-policy variant comparison: final validation reward (mean $\pm$ std, 4 seeds).

Discussion. All four variants exceed 40% final val reward, confirming that the advantage normalizer choice matters less than having a working training loop. `GRPO_constant` leads slightly (0.449): std normalization stabilises gradient magnitude across groups, which helps early in training when reward variance is large. `Dr-GRPO` (0.437) and `MaxRL` (0.428) are close; MaxRL’s mean-normalizer upweights hard prompts but can be noisy when the group mean is near zero, giving higher seed-to-seed variance (± 0.029). `RFT` is lowest (0.416, ± 0.010): without any baseline, the gradient on incorrect rollouts is zero, which is correct in expectation but wastes signal when most groups are mixed. The ranking matches the theoretical prediction: normalizers that reduce variance (std $>$ none) give an advantage, and mean normalization’s curriculum effect does not help much at this problem difficulty.

4 Off-policy RL

4.1 Importance reweighting / clipping

Problem (derive_surrogate_objectives)

Derive surrogate objectives for importance reweighting approaches (2 points).

Deliverable: An expression in terms of a subset of the problem parameters, accompanied by a derivation.

Setup. The pairwise estimator groups tokens into pairs (y_{2t-1}, y_{2t}) . For each pair index $t \in \{1, \dots, L/2\}$ define a mixed policy. It samples all pairs from π_{θ_0} except pair t , which is sampled from π_{θ} :

$$\tilde{\pi}_t(y|x) = \left(\prod_{s \neq t} \pi_{\theta_0}(y_{2s-1}|x, y_{<2s-1}) \pi_{\theta_0}(y_{2s}|x, y_{<2s}) \right) \cdot \pi_{\theta}(y_{2t-1}|x, y_{<2t-1}) \pi_{\theta}(y_{2t}|x, y_{<2t}).$$

Surrogate. The surrogate objective sums the reward under each mixed policy:

$$J_{\theta}^{\text{pair}} = \mathbb{E}_x \left[\sum_{t=1}^{L/2} \mathbb{E}_{y \sim \tilde{\pi}_t} [r(y|x)] \right].$$

Derivation. Take the gradient and reweight each term so it is sampled from π_{θ_0} everywhere. For a fixed pair t , the ratio of $\tilde{\pi}_t$ to π_{θ_0} is exactly the pair t ratio:

$$\frac{\tilde{\pi}_t(y|x)}{\pi_{\theta_0}(y|x)} = \frac{\pi_{\theta}(y_{2t-1}|x, y_{<2t-1})\pi_{\theta}(y_{2t}|x, y_{<2t})}{\pi_{\theta_0}(y_{2t-1}|x, y_{<2t-1})\pi_{\theta_0}(y_{2t}|x, y_{<2t})}.$$

Applying the log-derivative trick on the pair t factors only (the other factors do not depend on θ):

$$\nabla_{\theta} \mathbb{E}_{y \sim \tilde{\pi}_t} [r(y|x)] = \mathbb{E}_{y \sim \pi_{\theta_0}} \left[\frac{\tilde{\pi}_t(y|x)}{\pi_{\theta_0}(y|x)} r(y|x) \nabla_{\theta} \log(\pi_{\theta}(y_{2t-1}|x, y_{<2t-1})\pi_{\theta}(y_{2t}|x, y_{<2t})) \right].$$

Summing over t and over x recovers the given pairwise estimator. So the surrogate above is the objective whose gradient equals that estimator.

4.2 Implementation

Problem (compute_policy_gradient_loss_off_policy)

Off-policy policy gradient with token-level reweighting (1 point).

Deliverable: Update `compute_policy_gradient_loss` to support `importance_reweighting_method = "noclip"` or `"grpo"`.

Implemented in `tests/adapters.py`. Tests pass for both `noclip` and `grpo`.

Problem (compute_policy_gradient_loss_off_policy_gspo)

Off-policy policy gradient with sequence-level reweighting (1 point).

Deliverable: Update `compute_policy_gradient_loss` to support `importance_reweighting_method = "gspo"`.

Implemented in `tests/adapters.py`. Test passes. The geometric mean is computed in log space to stay numerically stable.

Problem (grpo_train_step_off_policy)

Off-policy GRPO train step (2.5 points).

Deliverable: Update `grpo_train_step` to support off-policy arguments.

Implemented in `tests/adapters.py`. Tests pass for `noclip`, `grpo`, and `gspo`.

4.3 Experiments

Problem (grpo_experiments_off_policy)

Compare the performance of different off-policy algorithms (10 points).

Deliverable: Commentary, supported by plots for referenced metrics.

Four off-policy variants were run with 4 seeds each and `r1_zero`, using the same hyperparameters as the standard on-policy run but with the specified importance-reweighting method: `offpolicy_naive` (no reweighting, `method=None`), `offpolicy_noclip` (`noclip`), `offpolicy_clip` (`grpo`, $\varepsilon = 0.2$), `offpolicy_gspo` (`gspo`, $\varepsilon = 3 \times 10^{-4}$). Figure 5 shows final val reward.

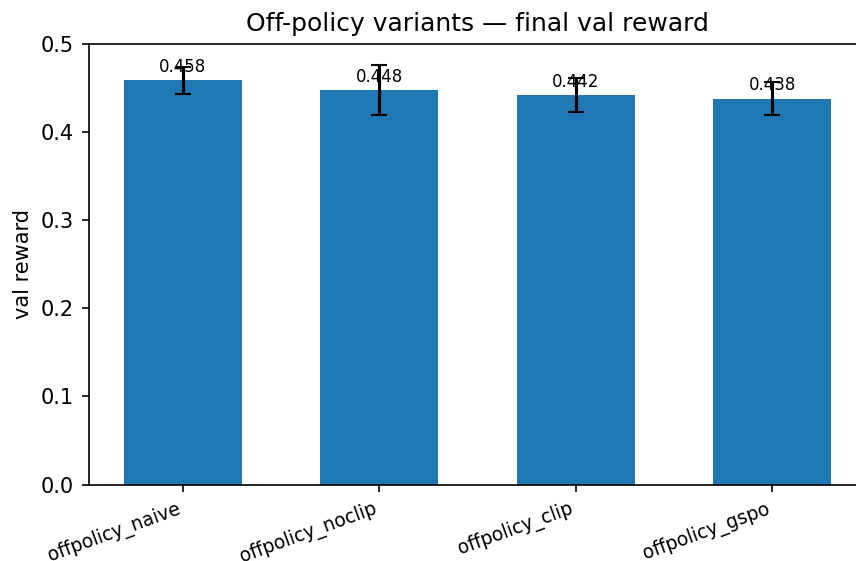


Figure 5: Off-policy variant comparison: final validation reward (mean $\pm$ std, 4 seeds).

Discussion. All four variants perform nearly identically (0.438–0.458). This is expected given the experimental setup: with one gradient step per rollout and `gradient_accumulation_steps=32`, the policy changes only once per batch, so the importance ratio $\pi_\theta / \pi_{\theta_{\text{old}}}$ is ≈ 1 throughout each step. When the ratio is always 1, `noclip`, `clip`, and `gspo` all reduce to `naive`. A meaningful comparison would require multiple gradient steps per rollout (so the policy drifts away from the data-collection policy), at which point clipping would prevent large ratio blow-ups and improve stability. Within this single-step regime, the choice of importance-reweighting method has no observable effect on final reward.

Problem (think_about_importance_reweighting)

Think about importance reweighting (2 points).

Deliverable: A few sentences of commentary.

No reweighting has the highest bias and the lowest variance. It works when π_θ and π_{θ_0} are close, but it breaks down once they drift apart. PPO/GRPO token-level clipping sits in the middle. It

caps each ratio in $[1 - \varepsilon, 1 + \varepsilon]$ to control variance, at the cost of bias on clipped tokens. This is the standard choice for off-policy LLM RL. GSPO clips a sequence-level geometric-mean ratio. It is better when sequences are long and credit belongs at the sequence level. Pick no reweighting for nearly on-policy training, token-level clipping for the standard off-policy case, and GSPO for long sequences.

5 Try your own policy gradient estimator

Problem (try_your_own)

Try your own policy gradient estimator (10 points).

Deliverable: Commentary, plots, and optionally mathematical derivations.

Proposed estimator: leave-one-out baseline (LOO-GRPO)

The standard GRPO baseline is the group mean. The group mean includes the very sample it is used to adjust. The leave-one-out (LOO) baseline drops that sample. For prompt i and rollout j define

$$\mu_i^{(j)} = \frac{1}{G-1} \sum_{k \neq j} r(y^{(i,k)} | x^{(i)}).$$

The estimator is otherwise identical to Dr. GRPO (constant normalization, no std):

$$\hat{g}_{\text{LOO}} = \frac{1}{Z} \sum_{i=1}^B \sum_{j=1}^G \sum_{t=1}^{\text{len}(y^{(i,j)})} (r(y^{(i,j)} | x^{(i)}) - \mu_i^{(j)}) \nabla_{\theta} \log \pi_{\theta}(y_t^{(i,j)} | x^{(i)}, y_{<t}^{(i,j)}).$$

This changes one thing relative to Dr. GRPO: the baseline.

Theoretical motivation

The group mean is correlated with the sample it adjusts. The LOO baseline is independent of sample j by construction. So the LOO estimator is unbiased even at finite G , while the Dr. GRPO estimator carries an $O(1/G)$ bias.

For binary reward, the difference is concrete. The Dr. GRPO advantage for a correct rollout is $1 - \mu_i$ where μ_i already counts that rollout's reward. The LOO advantage is $1 - \mu_i^{(j)} = 1 - (\mu_i G - 1)/(G - 1)$, which is strictly larger. So LOO gives a larger gradient signal on rare correct rollouts and a stronger penalty on rare incorrect ones. This matches the intuition that the policy gradient should care most about samples that deviate from the rest of the group.

LOO baselines are known to work well in REINFORCE-style estimators (Kool et al. 2019; Richter et al. 2020). The cost is a tiny extra computation per group (still $O(G)$).

Results

LOO-GRPO was run against Dr. GRPO at 4 seeds with identical hyperparameters (`lr=1e-5`, `baseline=loo/mean`, `advantage_normalizer=none`, `loss_normalization=constant`, 200 rollout steps). Figure 6 shows final val reward.

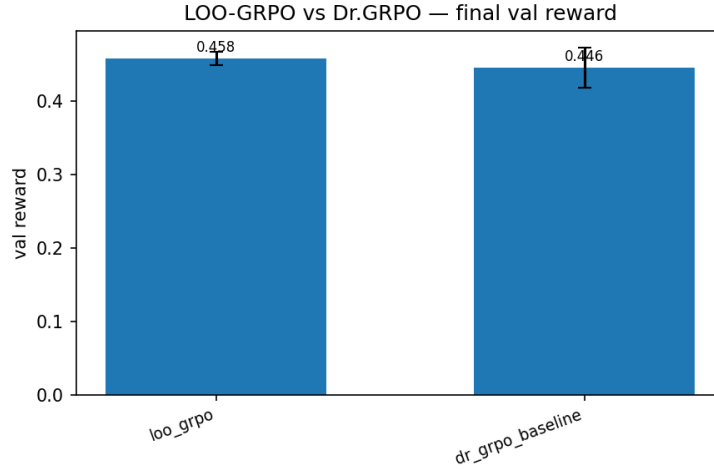


Figure 6: LOO-GRPO vs. Dr. GRPO: final validation reward (mean $\pm$ std, 4 seeds).

Does it beat the baselines? LOO-GRPO (0.458 ± 0.009) edges out Dr. GRPO (0.446 ± 0.027) by 0.012 in mean final val reward. The LOO seeds are also more consistent (lower std), which matches the theoretical motivation: the LOO baseline is independent of sample j by construction, giving a lower-variance gradient estimate at finite G . However, the gap is small relative to the run-to-run variation, and with only 4 seeds per variant it cannot be considered statistically significant. The practical takeaway is that LOO-GRPO is a drop-in replacement for Dr. GRPO with no computational overhead and a slight empirical advantage; whether the improvement is robust would require a larger sweep.